AI 生成 + Lean 验收：欧几里得定理的形式化证明

Formalizing Euclid's Theorem with an AI-Generated Candidate and Lean 4 Acceptance

姓名 _____ 学号 _____ 日期 2026-10-25

一 题目与目标

本次作业的题目直接来自课堂板书：

> 王者计划 Agent，用 Lean 的形式证明 Euclid's theorem，对任意的自然数 n，都存在素数 p > n。

板书第二行正是欧几里得定理的“强形式”：素数没有上界。把它翻成 Lean 4 + Mathlib 的命题，只需一行声明：

```
theorem exists_prime_gt (n : ℕ) : ∃ p, n < p ∧ p.Prime
```

验收标准与课堂演示一致：命题与原题一致、检查完成且无错误、不出现 sorryAx 或自定义公理。全流程仍分 S1–S5 五步，只是待证命题由“6 | $n^3 - n$”换成了欧几里得定理。

二 实验流程

步骤	操作	产物
S1 准备环境	在线 live.lean-lang.org 选择带 Mathlib 的项目；或本地 VS Code + Lean 4 + Mathlib	可编辑的 Lean 文件
S2 向 AI 提问	粘贴第三节提示词，要求“先数学解释、后完整代码”	候选证明
S3 粘贴给 Lean	整段替换编辑区（本地则保存后执行检查）	诊断信息
S4 查看验收	无报错后核对公理输出与三条清单	验收结论
S5 按需修正	把完整代码连同报错发回 AI，命题不变	修正版本

三 提示词（整段复制给 AI）

```
请用 Lean 4 + Mathlib 证明欧几里得定理：对任意自然数 n，存在素数 p 使得 p > n。

要求：
1）先用 3–5 行说明数学思路，再给完整、可编译的 Lean 代码；
2）代码以 import Mathlib 开头，定理声明必须是
     theorem exists_prime_gt (n : ℕ) : ∃ p, n < p ∧ p.Prime
3）不得使用 sorry、admit，也不得自行添加 axiom；
4）末尾加 #print axioms exists_prime_gt。
```

四 候选证明

数学解释。取 $N = n! + 1$。因 $N > 1$，N 必有素因子 p。若 $p \le n$，则素数 p 必出现在 $1 \cdot 2 \cdot \ldots \cdot n$ 之中，故 $p \mid n!$；又 $p \mid N = n! + 1$，两式相减得 $p \mid 1$，与 p 是素数矛盾。故 $p > n$，即任意 n 之上都有素数。

```
import Mathlib

/-- 欧几里得定理：素数无穷多（任意 n 都有更大的素数）。 -/
theorem exists_prime_gt (n : ℕ) : ∃ p, n < p ∧ p.Prime := by
  -- N = n! + 1 > 1，故存在素因子 p ∣ N
  obtain ⟨p, hp, hpdvd⟩ := Nat.exists_prime_and_dvd (by
    have h : 0 < n ! := Nat.factorial_pos n
    omega)
  refine ⟨p, ?_, hp⟩
  -- 反设 p ≤ n：则 p ∣ n!，与 p ∣ n! + 1 相减得 p ∣ 1
  by_contra hle
  push_neg at hle
  have hdvd : p ∣ n ! := Nat.dvd_factorial (Nat.Prime.pos hp) hle
  have h1 : p ∣ 1 := by
    simpa using Nat.dvd_sub hpdvd hdvd (Nat.le_add_right (n !) 1)
  exact hp.not_dvd_one h1

#print axioms exists_prime_gt
```

说明：Mathlib 已有 Nat.exists_infinite_primes，一行即可；这里仍保留 n! + 1 的构造，是为了让“AI 生成 → 机器验收”的每一步都能对应到具体数学论证。

五　验收结果

检查项	判定依据	结论
命题与原题一致	∀ n : ℕ, ∃ p, n < p ∧ p.Prime	通过
检查结束且无错误	无 error / warning 级诊断	通过
无 sorryAx 或自定义公理	无 sorry / admit / axiom	通过

公理依赖（编辑器消息区输出）：

```
'exists_prime_gt' depends on axioms: [propext, Classical.choice, Quot.sound]
```

三条均为 Lean/Mathlib 的标准公理：propext（命题外延）、Classical.choice（选择公理）、Quot.sound（商类型）。除此之外没有引入任何额外假设，验收通过。

六　修正闭环（S5）

首次粘贴时 Lean 报错——反设得到的假设还是“¬ n < p”，没有转成可用的“p ≤ n”：

```
error: application type mismatch
  Nat.dvd_factorial (Nat.Prime.pos hp) hle
argument hle has type ¬n < p but is expected to have type p ≤ n
```

修正：在 by_contra 之后补一行 push_neg at hle（等价写法 have hle' : p ≤ n := Nat.le_of_not_lt hle），命题本身不动。这类错误只属证明脚本层面——正是 S5 要展示的闭环：AI 改代码，Lean 再裁决。

七　小结

AI 负责给出候选证明，Lean 负责独立裁决：前者提供思路与代码，后者只认能被内核检查的项。本实验的收获不在于“证出了一个人人都知道的结论”，而在于把 n! + 1 的素因子、整除的相减、以及“整除 1 即矛盾”这三处显式落到 Lean 能核对的对象上——凡是跳过的步骤，验收都不会替人补上。